Java Process Table Function (PTF) User-Defined Function (UDF) examples — row-driven PTFs (set semantics and row semantics)

> This package contains **two row-driven** Process Table Function examples that together illustrate the two `ArgumentTrait` modes Flink offers for table arguments. **Both PTFs are row-driven**: every state transition or output emission is triggered exclusively by an incoming row. Neither uses timers, neither has an `onTimer()` callback, and neither reacts to the passage of time. (For the timer-driven counterparts, see `ptf_udf_timer_driven`.)
>
> Where the two UDFs differ is the **argument semantics** of their input table:
>
> - **`UserEventEnricher`** (set semantics, row-driven) — a stateful keyed operator that maintains per-user state across events. Uses `@ArgumentHint(ArgumentTrait.SET_SEMANTIC_TABLE)` and requires `PARTITION BY` in the SQL invocation.
> - **`OrderLineExpander`** (row semantics, row-driven) — a stateless one-to-many transformation that explodes a single order row into multiple line-item rows. Uses `@ArgumentHint(ArgumentTrait.ROW_SEMANTIC_TABLE)` and forbids `PARTITION BY`.
>
> Row-driven vs. timer-driven and set-semantic vs. row-semantic are **two orthogonal axes**. The directory name (`ptf_udf_row_driven`) refers to the first axis (no timers); the per-UDF qualifier in each section heading refers to the second (table-argument semantics).
>
> Both PTFs ship in the same fat JAR and are registered as separate functions from the same artifact. Together they demonstrate how the Process Table Function (PTF) API in Flink 2.1+ enables building both fully stateful operators and stateless table-valued transformations in Java that are directly callable from SQL.

Table of Contents

- **1.0 State and operators in a Process Table Function**
  - **1.1 What is state?**
  - **1.2 How operators use state**
  - **1.3 How `PARTITION BY` connects SQL to state**
  - **1.4 The role of `@StateHint` and `@ArgumentHint`**
- **2.0 UDF 1: User Event Enricher (set semantics, row-driven)**
  - **2.1 Enrichment logic**
  - **2.2 How it works end-to-end**
  - **2.3 Key concepts illustrated**
- **3.0 UDF 2: Order Line Expander (row semantics, row-driven)**
  - **3.1 What is row semantics?**
  - **3.2 Expansion logic**
  - **3.3 When to choose row semantics**
- **4.0 Comparing the UDFs in this package**
- **5.0 Resources**

1.0 State and operators in a Process Table Function

A **Process Table Function (PTF)** is a new category of user-defined function introduced in Flink 2.1. Unlike scalar or aggregate UDFs, a PTF is a **stateful operator** — it sits inside the Flink dataflow graph just like a built-in operator (e.g., a windowed aggregation or a keyed process function), but you define its logic in a plain Java class.

1.1 What is state?

In stream processing, **state** is data that persists across events. Without state every event is processed in isolation — you can filter or transform it, but you cannot count things, detect sequences, or remember what happened before. State is what turns a stateless pipe into an intelligent processor.

Flink manages state for you: it stores it in a **state backend** (heap, RocksDB, etc.), checkpoints it for fault tolerance, and restores it on failure. You never serialize or recover state manually.

1.2 How operators use state

Every Flink operator that needs memory between events holds state. A few examples:

Operator	State it keeps
Keyed window aggregation	Partial aggregates per key per window
Interval join	Buffered rows from both sides within the join window
Deduplication	Set of seen keys
ProcessTableFunction (PTF)	Whatever you declare via `@StateHint` — one instance per partition key

The PTF is special because **you** decide what the state looks like. You declare a POJO, annotate it with `@StateHint`, and Flink handles the rest — creating one instance per partition key, persisting it in the state backend, and checkpointing it automatically.

1.3 How `PARTITION BY` connects SQL to state

When you write:

```
SELECT *
FROM TABLE(
    UserEventEnricher(
        input => TABLE user_events PARTITION BY user_id
    )
)
```

Flink does the following behind the scenes:

1. **Key the stream** — `PARTITION BY user_id` tells the Flink planner to hash-partition the input by `user_id`, exactly like a `keyBy()` in the DataStream API.

2. **Allocate isolated state** — for each distinct `user_id`, Flink creates a separate `UserState` POJO instance. For example, as shown in the sample data, Alice's state never overlaps with Bob's.
3. **Route events** — every incoming row is routed to the partition (and therefore the state instance) that matches its `user_id`.
4. **Call `eval()`** — your `eval()` method receives the row *and* the correct state instance already loaded. You read and mutate the POJO fields directly — no `ValueState.get()` / `.update()` boilerplate.
5. **Checkpoint** — Flink snapshots all state instances periodically. On failure it restores them and replays from the last checkpoint, so your PTF is exactly-once by default.

1.4 The role of `@StateHint` and `@ArgumentHint`

These two annotations are the bridge between your Java code and the Flink operator graph:

- `@StateHint` — marks a parameter as operator state. Flink sees the annotated POJO and wires it into the keyed state backend. Each `PARTITION BY` key gets its own instance, automatically serialized, checkpointed, and restored.
- `@ArgumentHint(ArgumentTrait.SET_SEMANTIC_TABLE)` — declares that the input argument is a *set-semantic table*, meaning the PTF acts as a keyed, stateful virtual processor over the entire partitioned stream. This is what distinguishes a PTF from a simple row-at-a-time scalar UDF.

Together, these annotations let you write what *looks* like a plain method but *executes* as a fully fault-tolerant, distributed, keyed-state operator inside the Flink pipeline.

2.0 UDF 1: User Event Enricher (set semantics, row-driven)

This example puts the above concepts into practice. The `UserEventEnricher` PTF reads raw user-interaction events from a Kafka topic (`user_events`), enriches each event with session and counting information, and writes the result to a second Kafka topic (`enriched_events`).

2.1 Enrichment logic

For every incoming event the function maintains three pieces of **per-user state** (one state instance per `PARTITION BY user_id` key):

State field	Purpose
`sessionId`	Monotonically increasing session counter. Incremented each time a `"login"` event arrives.
`eventCount`	Running count of events **within the current session**. Reset to zero on each new session, then incremented for every event (including the login itself).
`lastEvent`	The `event_type` of the most recent event for that user.

Each incoming row is emitted immediately with these three extra fields appended, so the output schema is:

```
user_id     STRING    — passed through automatically via PARTITION BY
event_type  STRING
payload     STRING
```

```
session_id  BIGINT    — which session this event belongs to
event_count BIGINT    — position of this event within the session
last_event  STRING    — the event type just processed
```

2.2 How it works end-to-end

```
Kafka (user_events)
        │
        ▼
  ┌──────────────┐
  │ user_events │   Flink SQL source table (JSON / latest-offset)
  └──────────────┘
        │
        ▼
  UserEventEnricher(input => TABLE user_events PARTITION BY user_id)
        │
        │  Per-user stateful enrichment:
        │    • login → new session, reset count
        │    • any event → increment count, record last_event
        │
        ▼
  ┌─────────────────┐
  │ enriched_events│   Flink SQL sink table → Kafka (enriched_events)
  └─────────────────┘
```

2.3 Key concepts illustrated

- `ProcessTableFunction` — the Flink 2.x API for stateful, set-semantic UDFs callable from SQL.
- `@StateHint` — declares a POJO whose fields Flink automatically persists per partition key, eliminating manual `ValueState` / `MapState` boilerplate.
- `@ArgumentHint(ArgumentTrait.SET_SEMANTIC_TABLE)` — tells Flink the input is a keyed, stateful virtual processor (set semantics), not a simple row-at-a-time scalar function.
- `PARTITION BY` — the SQL-side mechanism that keys the input table so each `user_id` gets its own isolated state instance.

3.0 UDF 2: Order Line Expander (row semantics, row-driven)

The second PTF in this package, `OrderLineExpander`, is also row-driven — every output is triggered exclusively by an incoming row, with no timers involved. Where it differs from `UserEventEnricher` is the **table-argument semantics**: the enricher uses set semantics and is a fully stateful keyed operator, while the expander uses row semantics and is a stateless one-to-many transformation that processes every row in complete isolation.

3.1 What is row semantics?

When you mark a table argument with `@ArgumentHint(ArgumentTrait.ROW_SEMANTIC_TABLE)`, you are telling Flink:

> "Each row in this table can be processed in complete isolation. There is no correlation between rows, no shared state, and no required ordering."

This gives the Flink runtime maximum freedom:

1. **Free distribution** — rows can be routed to any virtual processor in any order; the framework parallelizes without constraints.
2. **No state backend involvement** — no checkpointing of operator state, no key groups, no rebalancing on rescale.
3. **No `PARTITION BY` clause** — the SQL caller passes the table directly with no keying.
4. **`eval()` sees only the current row** — there is no `@StateHint` parameter, no `Context` reference to keyed state, and no timers.

The PTF behaves like an enhanced table-valued function: it can emit zero, one, or many output rows per input row, but cannot remember anything from prior rows. Flink's planner will reject any attempt to add `@StateHint` to a row-semantic PTF.

3.2 Expansion logic

The `OrderLineExpander` accepts a table of orders where each row contains a comma-separated list of items and a positionally aligned comma-separated list of quantities. For every input row, it emits **one output row per item**, expanding the order into individual line items.

Input schema (the `orders` table):

```
order_id    STRING    — unique order identifier
customer    STRING    — customer name
items       STRING    — comma-separated item names ("widget,gadget,gizmo")
quantities  STRING    — comma-separated quantities  ("2,1,5")
```

Output schema (one row per item):

```
order_id     STRING
customer     STRING
item_name    STRING    — a single item from the items list
quantity     INT       — the corresponding quantity (defaults to 1 if
missing)
line_number  INT       — 1-based position of this item in the order
```

Example — a single input row:

order_id	customer	items	quantities
`O-100`	`Alice`	`widget,gadget,gizmo`	`2,1,5`

produces three output rows:

order_id	customer	item_name	quantity	line_number
O-100	Alice	widget	2	1
O-100	Alice	gadget	1	2
O-100	Alice	gizmo	5	3

The function tolerates malformed input gracefully: empty or null `items` produce no output, and missing/non-numeric `quantities` default to `1`.

SQL invocation:

```
SELECT *
FROM TABLE(
    OrderLineExpander(
        input => TABLE orders
    )
);
```

Notice there is **no `PARTITION BY`** clause — row semantics forbid partitioning.

Source: `OrderLineExpander.java`

3.3 When to choose row semantics

Use `ROW_SEMANTIC_TABLE` when:

- You need to **explode** one row into many (this example).
- You are performing a **stateless transformation** more complex than a scalar UDF can express (e.g. multi-column derivation, conditional emission).
- You want to **filter** rows with logic too complex for a `WHERE` clause.
- You are **enriching** rows from a self-contained computation that needs no memory of prior rows.

Use `SET_SEMANTIC_TABLE` instead when you need to count, deduplicate, detect sequences, maintain sessions, or otherwise correlate information across rows — as `UserEventEnricher` does.

4.0 Comparing the UDFs in this package

Both UDFs are **row-driven** (no timers); the table below compares them along the *other* axis — table-argument semantics.

Aspect	`UserEventEnricher` (set semantics, row-driven)	`OrderLineExpander` (row semantics, row-driven)
`@ArgumentHint` trait	`SET_SEMANTIC_TABLE`	`ROW_SEMANTIC_TABLE`

Aspect	`UserEventEnricher` (set semantics, row-driven)	`OrderLineExpander` (row semantics, row-driven)
Trigger for `eval()`	Incoming row (no timers)	Incoming row (no timers)
`PARTITION BY` in SQL	**Required**	**Forbidden**
State (`@StateHint`)	Allowed and central to the design	**Not allowed**
Per-key isolation	One state instance per partition key	N/A — rows are independent
Checkpointing	Full state snapshot per checkpoint	Nothing to checkpoint
`eval()` parameters	`Context` + `@StateHint` POJO + input row	Just the input row
Output cardinality	Typically one per input	Zero, one, or many per input
Parallelism model	Hash-partitioned by key	Free row distribution
Use cases	Sessionization, counting, deduplication, sequence detection	Exploding rows, complex stateless transforms, content-based filtering

The two traits are not interchangeable: choosing the right one is a *design* decision, not a *tuning* decision. Row semantics give you stateless table-valued functions; set semantics give you fully fault-tolerant keyed operators. Both PTFs in this package compile into the same fat JAR and are registered as separate Flink functions via two `CREATE FUNCTION ... USING JAR` statements that point to the same artifact but reference different fully-qualified class names (`ptf.UserEventEnricher` and `ptf.OrderLineExpander`).

5.0 Resources

- Apache Flink User-defined Functions
- Create a User-Defined Function with Confluent Cloud for Apache Flink
- Process Table Functions in Confluent Cloud for Apache Flink